

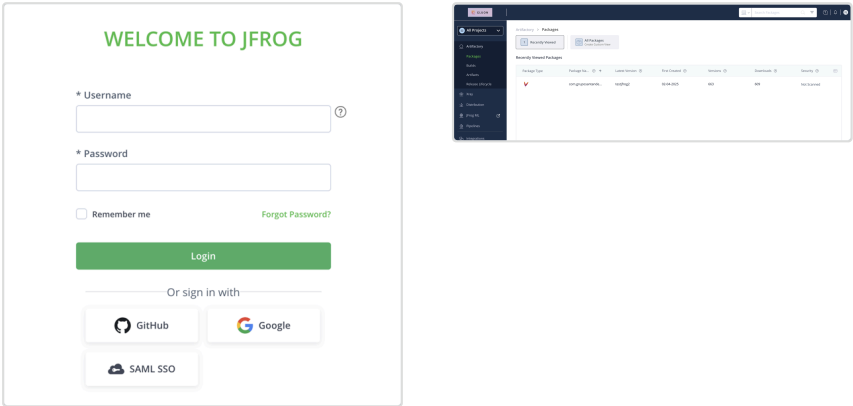
Maven repositories and JFrog

- 1 [Login](#)
- 2 [Search](#)
- 3 [Get a token and configure Maven Repository](#)
- 4 [Potential issues](#)
 - 4.1 [MAVEN_OPTS](#)
 - 4.2 [PKIX path building failed](#)
 - 4.3 [Testcontainer / Docker error](#)
 - 4.4 [Maven Repository configuration issue](#)
 - 4.5 [Netskope blocking issue](#)
 - 4.6 [External contractors without admin permissions](#)

⚠ These changes are available from version `12.21.25` / `13.1.50` of the mb-parent-pom.

Login

Access the repository [here](#) and click on the **SAML SSO** sign-in option. If connected to the VPN, you are automatically logged in and directed to the main page.

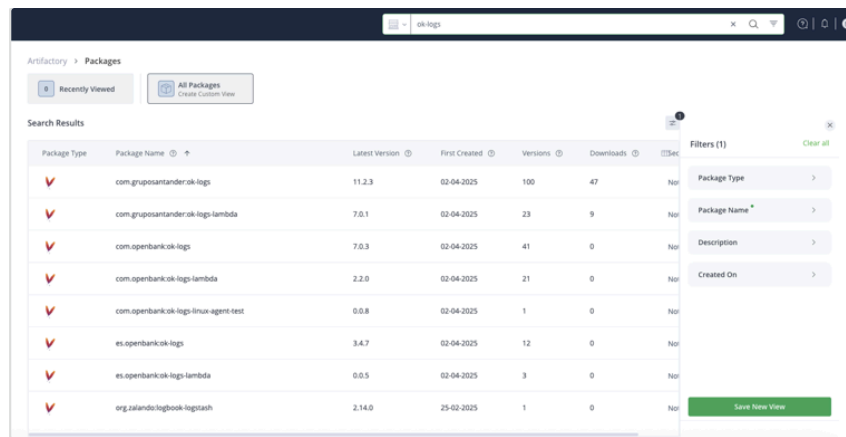


Search

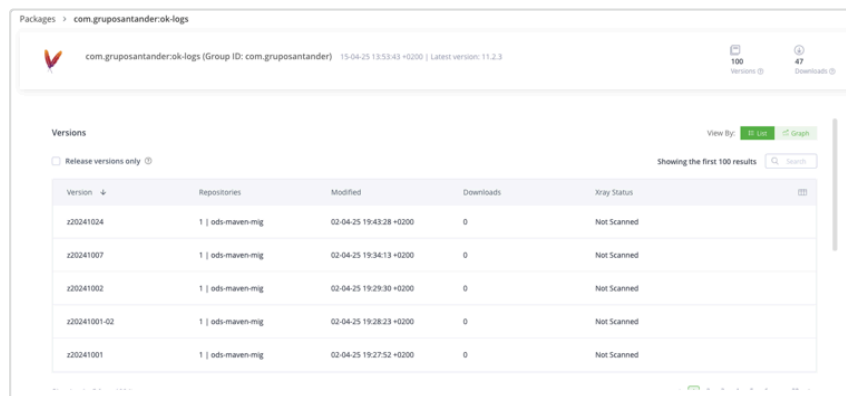
- 1. To search for an artifact, click on the search box located in the upper-right corner and enter the name of the package.



- 2. The results are then shown.

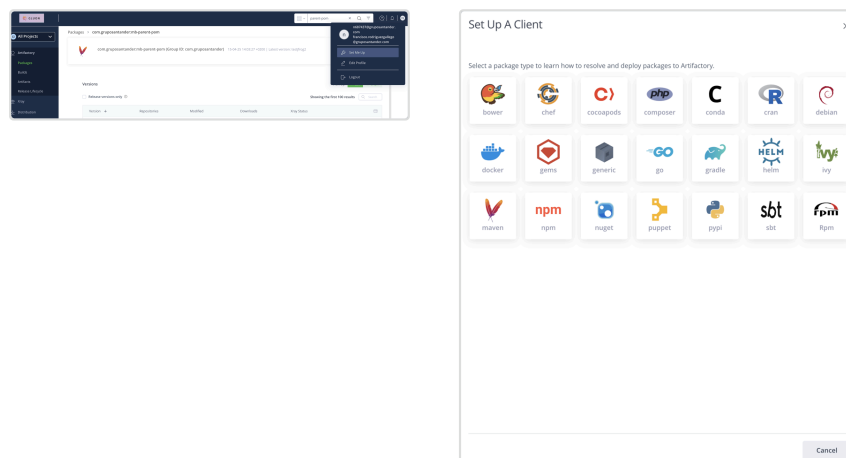


3. Click on the desired result—such as `com.gruposantander:ok-logs`—to view the information.

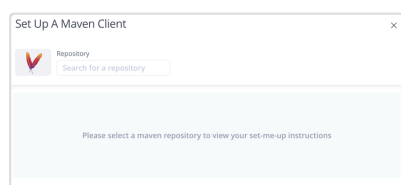


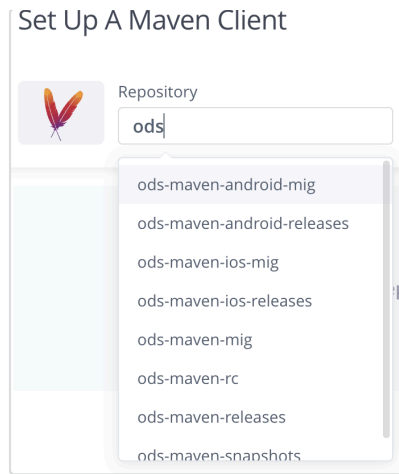
Get a token and configure Maven Repository

1. Click on the N icon in the upper-left corner and select **Set Me Up**. Select **Maven** in the new tab that opens.

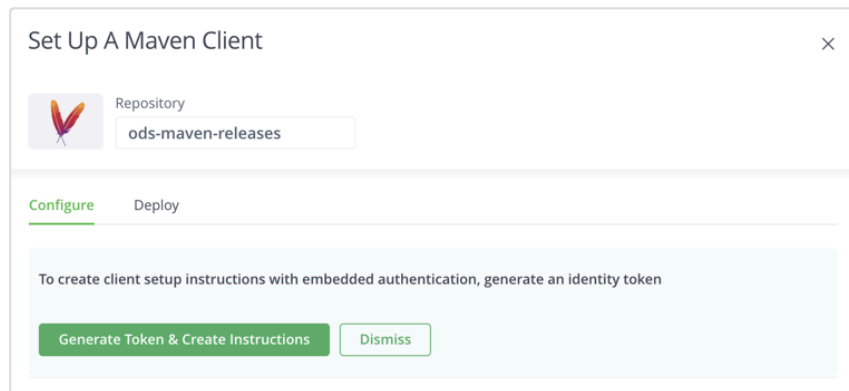


2. A new tab appears with a text box to **Search for a repository**. Begin typing `ods` and the relevant repositories start to appear.





3. For Java, select **ods-maven-releases** and proceed to the next step.



4. Click on **Generate Token & Create Instructions**.

A token is generated along with instructions to configure the **settings.xml** file. Be sure to change the selectable options to **ods-maven-public** as shown below.

Set Up A Maven Client

Repository

ods-maven-releases

Configure
Deploy

The token has been generated successfully!

Copy

To configure your Maven client, create the `settings.xml` file, and place it in one of the following locations according to your use case:

- User-specific install: `${user.home}/.m2/settings.xml`
- Global machine install: `${maven.home}/conf/settings.xml`
- Custom `settings.xml` file: Pass it as an argument during execution, for example: `mvn install -s settings.xml`

Releases
ods-maven-public

Snapshots
ods-maven-public

☐ Mirror Any
abi-maven-public

Plugin Releases
ods-maven-public

Plugin Snapshots
ods-maven-public

For more information, refer to our [documentation](#)

Generate Settings

5. Click on **Generate Settings**.

You see the `settings.xml` file and you can download it by clicking on **Download Snippet**.

Snippet
Download Snippet

> Deploy Settings

Copy

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <settings xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.2.0
3 http://maven.apache.org/xsd/settings-1.2.0.xsd"
4 xmlns="http://maven.apache.org/SETTINGS/1.2.0"
5 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
6   <servers>
7     <server>
8       <username>n687437@gruposantander.com</username>
9
10    <password>[REDACTED]/password>
11    <id>central</id>
12  </server>
13  <server>
14    <username>n687437@gruposantander.com</username>
15
16    <password>[REDACTED]/password>
17    <id>snapshots</id>
18  </server>
19 </servers>
20 <profiles>
21   <profile>
22     <repositories>

```

6. Copy the downloaded `settings.xml` file to `${user.home}/.m2/settings.xml` to start using it for Maven compilations.

Tip

If a message appears during project compilation indicating that **a library is already present in your local Maven repository**, it could be due to a previous download from a remote repository that is currently unavailable.

“The `org.opensaml:opensaml-xmlsec-impl:4.3.2` artifact is present in the local repository, but cached from a remote repository ID that is unavailable in the current build context, verifying that it is downloadable from [central (<https://gluoneurope.jfrog.io/artifactory/ods-maven-public>, default, releases), snapshots (<https://gluoneurope.jfrog.io/artifactory/ods-maven-public>, default, releases+snapshots)] Downloading from central: <https://gluoneurope.jfrog.io/artifactory/ods-maven-public/org/opensaml/opensaml-xmlsec-impl/4.3.2/opensaml-xmlsec-impl-4.3.2.pom>”

If these messages are disruptive, delete your local repository to download the dependencies again.

Potential issues

MAVEN_OPTS

- ✓ If you encounter difficulties connecting to the JFrog repository due to **certificate issues**, resolve this by creating an environment variable. Avoid repeating this process every time you open a console by adding it to your **bash(.bashrc)** or **zsh(.zshrc)** profile:

```
1 export MAVEN_OPTS="-Djavax.net.ssl.trustStore=$(echo $(/usr/libexec/java_home)/lib/security/cacerts) -Djavax.net.ssl.trustStorePassword=changeit"
```

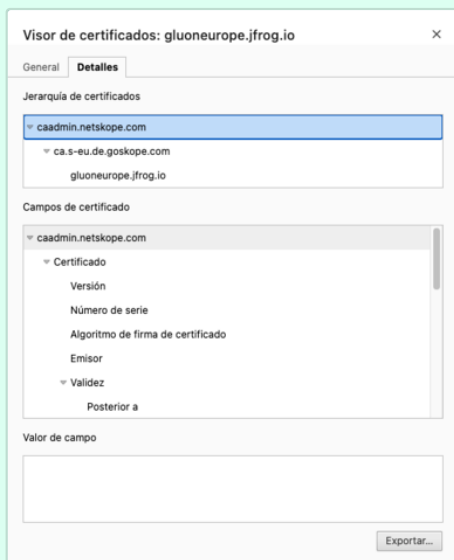
PKIX path building failed

- ✗ If you encounter the following error:

```
1 PKIX path building failed: sun.security.provider.certpath.SunCertPathBuilderException: unable to find valid certification path to requested target...
```

- ✓ Follow these steps:

1. Log in to <https://gluoneurope.jfrog.io/ui/packages> using SAML SSO.
2. Download the certificate:

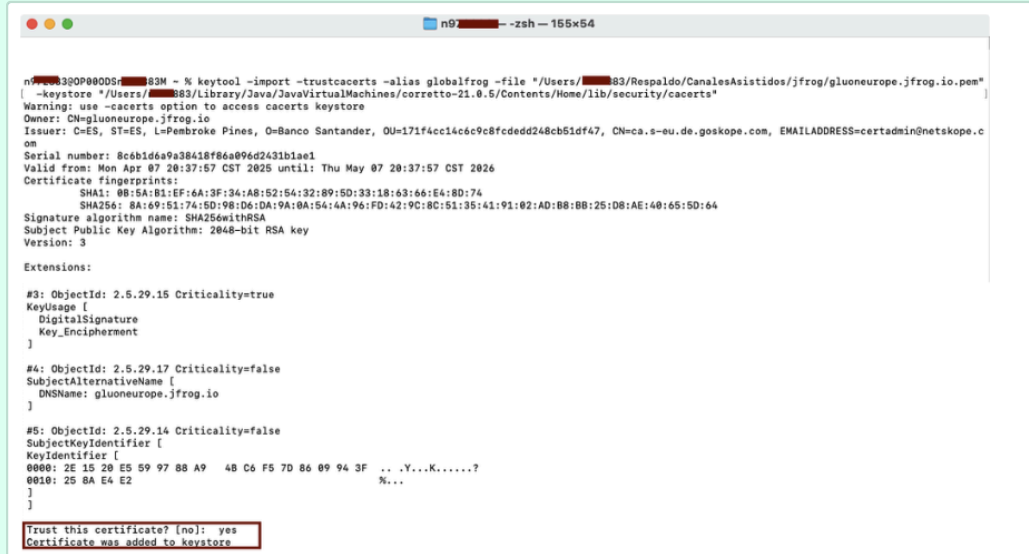


3. Install the certificate by executing the following command

```
1 keytool -import -alias jfrog -keystore $JAVA_HOME/lib/security/cacerts -file {path_to_cert}
```

You may need to set up `$JAVA_HOME` variable. Set it to your Java SDK Home. On ARM Macs it is usually at `/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home` —pay attention as the path is version specific.

Alternatively, use the whole path when invoking the `keytool` command.



```
n92 ~ % keytool -import -trustcacerts -alias globaljfrog -file "/Users/83/Respaldo/CanalesAsistidos/jfrog/gluoneurope.jfrog.io.pem"
Warning: use -cacerts option to access cacerts keystore
Owner: CN=gluoneurope.jfrog.io
Issuer: C=ES, ST=ES, L=Pembroke Pines, O=Banco Santander, OU=171f4cc14c6c9c8fcdedd248cb51df47, CN=ca.s-eu.de.goskope.com, EMAILADDRESS=certadmin@netskope.c
on
Serial number: 8c6b1da9a38418f86a096d2431b1ae1
Valid from: Mon Apr 07 20:37:57 CST 2025 until: Thu May 07 20:37:57 CST 2026
Certificate fingerprints:
  SHA1: 0B:5A:B1:EF:6A:3F:34:A8:52:54:32:89:5D:33:18:63:66:E4:8D:74
  SHA256: 8A:69:51:74:5D:98:D6:DA:9A:8A:54:4A:96:FD:42:9C:8C:51:35:41:91:02:AD:BB:BB:25:D8:AE:40:65:5D:64
Signature algorithm name: SHA256withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

Extensions:
#3: ObjectID: 2.5.29.15 Criticality=true
KeyUsage [
  DigitalSignature
  Key_Encipherment
]
#4: ObjectID: 2.5.29.17 Criticality=false
SubjectAlternativeName [
  DNSName: gluoneurope.jfrog.io
]
#5: ObjectID: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
  KeyIdentifier [
    0000: 2E 15 20 E5 59 97 88 A9 4B C6 F5 7D 86 09 94 3F ..Y...K.....?
    0010: 25 8A E4 E2                                     %...
  ]
]

Trust this certificate? (no): yes
Certificate was added to keystore
```

Testcontainer / Docker error

✖ Unauthorized errors occur when attempting to use Docker images:

```
1 10:32:59.641 [docker-java-stream--634239146] ERROR com.github.dockerjava.api.async.ResultCallbackTemplate --
Error during callback
2 com.github.dockerjava.api.exception.InternalServerErrorException: Status 500: {"message":"failed to resolve
reference \"gluoneurope.jfrog.io/ods-docker-registry/redis:7.0.11\": failed to authorize: failed to fetch
anonymous token: unexpected status from GET request to
https://gluoneurope.jfrog.io/artifactory/api/docker/ods-docker-registry/v2/token?scope=repository%3Aods-
docker-registry%2Fredis%3Apull&scope=repository%3Aredis%3Apull&service=gluoneurope.jfrog.io: 401
Unauthorized"}
```

✔ Resolve this issue by executing the following command:

```
1 docker login gluoneurope.jfrog.io
```

Ensure that the username and password you use are those configured in your `settings.xml`, as detailed [here](#).

Maven Repository configuration issue

✖ When working with Maven repositories, the repository URL auto-generated by JFrog may default to `zbr-maven-public`. For this project, the correct repository to use is `ods-maven-public`.

✔ Ensure that the Maven repository in your `settings.xml` is configured to use `ods-maven-public` instead of `zbr-maven-public`. Below is an example of the correct configuration:

```
1 <repositories>
2   <repository>
```

```

3     <id>ods-maven-public</id>
4     <url><https://jfrog.example.com/artifactory/ods-maven-public</url>>
5 </repository>
6 </repositories>

```

Netskope blocking issue

✖ When accessing Maven repositories, Netskope may block requests if the domain is **santandereurope** instead of **gluoneurope**. This can prevent successful dependency resolution during builds.

✔ Ensure that the Maven repository URL uses the **gluoneurope** domain instead of **santandereurope**. Below is an example of the correct configuration:

```

1 <repositories>
2   <repository>
3     <id>ods-maven-public</id>
4     <url>https://gluoneurope.jfrog.io/artifactory/ods-maven-public</url>
5   </repository>
6 </repositories>

```

1. Open your **settings.xml** file.
2. Check the **<repositories>** section for the repository URL.
3. Replace any occurrence of **santandereurope** with **gluoneurope**.
4. Save the changes and run your Maven build to ensure dependencies resolve correctly.

External contractors without admin permissions

✔ Download **JDK distribution** from **IntelliJ** (such as **corretto-21.0.7**) to your local path.

For example **C:\Users\x000000\jdk\corretto-21.0.7** (replace x000000 with your ID).

1. Click on the **Windows Icon** and type *Environment variables for this account*.
2. Include the following variables:
 - a. **JAVA_HOME** : **C:\Users\x000000\jdk\corretto-21.0.7** (or any other similar to this one).
 - b. **JAVA_ROOT** : **C:\Users\x000000\jdk** (or any other similar to this one).
 - c. **JAVAHOME** : **C:\Users\x000000\jdk\corretto-21.0.7** (or any other similar to this one).
3. Add them to the previous path: **C:\Users\x000000\jdk\corretto-21.0.7\bin**
4. Install the **gluoneurope.jfrog.io.crt** certificate.

```

1 "C:\Users\x000000\jdk\corretto-21.0.7\bin\keytool" -import -trustcacerts -alias globalfrog -file
  "C:\Users\x000000\Downloads\gluoneurope.jfrog.io.crt" -keystore "C:\Users\x000000\jdk\corretto-
  21.0.7\lib\security\cacerts"

```

5. Install the **caadmin.netskope.com.crt** certificate.

```

1 "C:\Users\x000000\jdk\corretto-21.0.7\bin\keytool" -import -trustcacerts -alias netskope -file
  "C:\Users\x000000\Downloads\caadmin.netskope.com.crt" -keystore "C:\Users\x000000\jdk\corretto-
  21.0.7\lib\security\cacerts"

```